

RESTAURANTOS MASTER SPECIFICATION V2.0

PART 3 — DATA ARCHITECTURE, FIREBASE, PUBLISHING, DATA FLOW & ENGINEERING EXECUTION

1. DATA ARCHITECTURE PHILOSOPHY

RestaurantOS is a multi-tenant SaaS application.

Every restaurant is completely isolated from every other restaurant.

No restaurant may read, write, modify or access another restaurant's data.

All data must belong to one Restaurant ID.

Every feature in the system must read and write data through the shared RestaurantData structure.

Never hardcode business information.

Never duplicate data.

Every module references the same data source.

2. SINGLE SOURCE OF TRUTH

The following information must exist only once:

Restaurant Name

Restaurant Logo

Business Description

Phone Number

Email

Address

Opening Hours

Currency

Timezone

Languages

Social Links

Brand Colors

Theme

Logo Variants

Website Domain

Reservation Settings

Delivery Settings

Tax Settings

These values are referenced throughout the system.

Never create duplicate copies.

3. DATA FLOW

Website Manager



RestaurantData



Validation



Draft Storage



Preview Environment



Publish



Customer Website

The customer website must never read directly from draft data.

Only published data should appear publicly.

4. DRAFT SYSTEM

Every editable module supports:

Draft

Published

Archived

Deleted

Never overwrite published data while editing.

Users edit drafts.

Publishing replaces the live version.

5. PUBLISHING WORKFLOW

Required workflow:

Edit



Autosave Draft



Preview



Validation



Publish Confirmation



Publish



Version Created



Live Website Updated

Publishing must validate:

Missing Images

Broken Links

Required Fields

SEO Errors

Accessibility Warnings

Empty Required Sections

Invalid URLs

Publishing should block critical errors.

Warnings may still allow publishing.

6. VERSION HISTORY

Every publish creates a version.

Each version stores:

Version Number

Timestamp

User

Summary

Changed Sections

Rollback Reference

Versions cannot be edited.

Rollback creates a new version.

7. AUTOSAVE

Autosave every editable field.

Maximum delay:

2–5 seconds after user stops typing.

Display:

Saving...

Saved

Offline

Retry

Conflict

Never interrupt typing.

8. IMAGE STORAGE

Every uploaded image belongs to Media Library.

Pages reference Media IDs.

Never duplicate image files.

Replacing an image automatically updates every location using that image if the user chooses that option.

Deleting an image currently in use must require confirmation.

9. MEDIA RULES

Every media file stores:

ID

Filename

Upload Date

Uploaded By

Alt Text

Caption

File Size

Width

Height

Usage Count

Folder

Tags

Visibility

Status

10. MENU DATA

Categories contain Menu Items.

Menu Items never contain Categories.

Categories reference Menu Items.

Menu Items reference Images.

Menu Items support:

Availability

Scheduling

Pricing

Variants

Tags

Nutrition

Allergens

Preparation Time

11. CUSTOMER DATA

Each customer stores:

Profile

Contact

Order History

Reservation History

Preferences

Marketing Consent

Loyalty

Notes

Tags

Blocked Status

No duplicated customer records.

12. ORDER DATA

Every order contains:

Customer

Items

Modifiers

Taxes

Discounts

Fees

Delivery Method

Payment Method

Payment Status

Order Status

Timeline

Notes

Receipts

Invoices

Order timeline is immutable.

Every status change creates a timeline event.

13. RESERVATION DATA

Reservation stores:

Customer

Guests

Date

Time

Duration

Assigned Table

Special Requests

Status

Reminder Status

Arrival Time

Completion Time

Cancellation Reason

14. WEBSITE PAGES

Every page stores:

ID

Slug

Title

SEO Title

SEO Description

Open Graph

Visibility

Published Status

Draft Status

Last Edited

Version

Author

Components

Pages are component-driven.

Never hardcode layouts.

15. COMPONENT DATA

Every section on every page is a reusable component.

Each component stores:

Type

Content

Layout

Visibility

Theme Overrides

Spacing

Animation

Media References

Links

Order

Never duplicate component logic.

16. WEBSITE NAVIGATION

Navigation references pages.

Never hardcode URLs.

Navigation supports:

Internal Links

External Links

Dropdowns

Visibility Rules

Icons

Ordering

Scheduling

17. USER ROLES

Owner

Administrator

Manager

Marketing

Reception

Kitchen

Editor

Viewer

Permissions are role-based.

Never hardcode permissions into components.

18. SECURITY

Every request validates:

Restaurant ID

Authenticated User

Role

Permission

Resource Ownership

Never expose hidden routes.

Never expose another tenant's data.

19. FIREBASE STRUCTURE

Each restaurant has independent collections.

Suggested structure:

Restaurant

Website

Menu

Reservations

Orders

Customers

Media

Analytics

Marketing

Settings

Users

Logs

Drafts

Versions

Notifications

Integrations

Avoid deeply nested documents where possible.

Keep collections modular.

20. FIREBASE RULES

Every document contains:

Restaurant ID

Created By

Created At

Updated By

Updated At

Status

Version

Never create anonymous documents.

21. LOGGING

Every major action creates a log.

Examples:

Published Website

Deleted Menu Item

Updated Reservation

Changed Theme

Uploaded Image

Deleted Customer

Changed Settings

Logs include:

Timestamp

User

Action

Affected Resource

Previous Value

New Value

22. NOTIFICATIONS

Notification types:

Order

Reservation

Website

Media

Marketing

Payments

System

Security

Each notification supports:

Read

Unread

Archive

Delete

Priority

23. API ARCHITECTURE

All modules communicate through shared services.

Avoid direct component-to-database communication.

Components call services.

Services handle business logic.

Services communicate with Firebase.

24. PERFORMANCE

Lazy load:

Images

Charts

Media

Large Tables

Analytics

Infinite lists should paginate.

Avoid loading entire collections.

25. CACHING

Cache:

Theme

Navigation

Published Website

Restaurant Settings

Media Metadata

Invalidate cache after publishing.

26. SEARCH

Search indexes:

Pages

Customers

Orders

Reservations

Media

Menu

Settings

Analytics

Results grouped by category.

27. IMPORT / EXPORT

Support:

Restaurant Settings

Menus

Media References

Reservations

Customers

Theme

Website Configuration

Never export sensitive credentials.

28. FUTURE PAYMENT SYSTEM

Payment architecture must be provider-independent.

Never build directly around one gateway.

Support future integration with:

Stripe

PayPal

Flutterwave

Paystack

Square

Other providers

Switching providers must not require rewriting order logic.

29. INTERNATIONALIZATION

Support:

Multiple Languages

Multiple Currencies

Multiple Timezones

Regional Date Formats

Regional Number Formats

Tax Variations

Never hardcode USD.

Never hardcode English-only assumptions.

30. DEPLOYMENT

Environment variables only.

Never commit secrets.

Development

Testing

Production

must remain separate.

31. ENGINEERING RULES

Never duplicate code.

Never create unnecessary components.

Never create page-specific components when a reusable component can be used.

Favor composition over duplication.

Every repeated UI element becomes a reusable component.

32. CODE QUALITY

Use strict TypeScript.

Use consistent naming.

Avoid unused files.

Avoid dead code.

Avoid commented-out code.

Avoid duplicate utilities.

Keep folders organized.

33. BUILD REQUIREMENTS

The application must build successfully without warnings or errors.

No broken routes.

No broken imports.

No missing assets.

No unused dependencies.

No placeholder pages in production.

34. AI EXECUTION RULES

Before generating code:

Read PROJECT.md.

Read Part 1.

Read Part 2.

Read this Part 3.

Audit the repository.

Audit reusable components.

Audit layouts.

Audit data structures.

Identify inconsistencies.

Generate a development plan.

Refactor shared architecture before expanding functionality.

Never regenerate completed work.

Never replace working components unnecessarily.

Improve consistency before adding new features.

35. FINAL ACCEPTANCE CRITERIA

RestaurantOS is considered production-ready only when:

All pages share a consistent design system.

All modules use the shared RestaurantData structure.

No hardcoded restaurant information exists.

No hardcoded demo images remain.

Every editable item supports Draft, Preview, Publish, and Version History.

The Website Manager and Customer Website remain synchronized through the shared data architecture.

The application is responsive across supported screen sizes.

The codebase is modular, maintainable, Firebase-ready, and free of build errors.

The finished product must feel like a commercial SaaS platform rather than an AI-generated project or a collection of disconnected pages.